

Programmation Logique

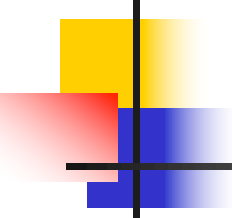
Khadim Dramé

kdrame@univ-zig.sn

Département Informatique

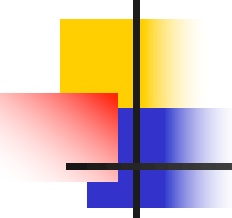
UASZ

Juillet 2025



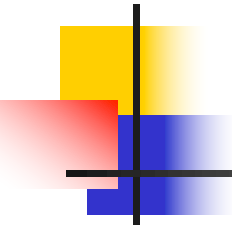
Introduction

- Prolog : Programmation logique
- Un langage de programmation déclarative qui reposant sur la logique des prédicats
- Il a été conçu vers les années 72
- Il a suscité un grand engouement vers les années 80
- Langage adapté au calcul symbolique et au raisonnement



Introduction

- Un programme Prolog est un ensemble de clauses exprimant des :
 - Faits
 - Règles
 - Questions
- Pas d'affectation
- Pas d'instructions
- Pas de boucles explicites



Syntaxe Prolog

- Prolog & Logique des prédicats

- Tous les humains sont mortels.

se traduit en

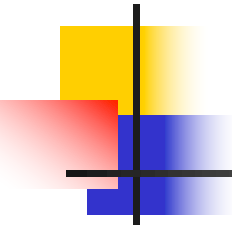
- Logique des prédicats par

$$\forall x (\text{Humain}(x) \rightarrow \text{Mortel}(x))$$

- Prolog par

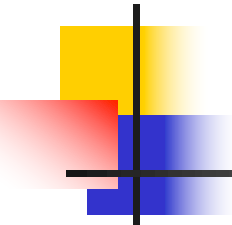
mortel(x):- humain(x).

Logique des prédicats	Prolog
$\rightarrow$	<code>:-</code>
$\wedge$	<code>,</code>
$\vee$	<code>;</code>
$\neg$	<code>not</code>



Syntaxe Prolog

- Constante
 - Entiers, flottants et chaînes de caractères
 - Commence par une **minuscule**
- Variable
 - Chaîne alphanumériques commençant par une **majuscule** ou un **_**
- Terme
 - Constante, variable ou un symbole fonctionnel appliqué à une liste de termes
- NB: Prolog est sensible à la casse



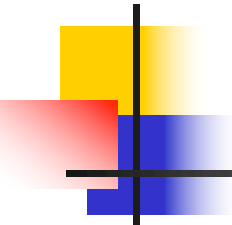
Syntaxe Prolog

■ Littéral

- Symbole de prédicat appliqué à une liste de termes
- Exprime une propriété vraie ou fausse

■ Clause

- Suite de littéraux composée
 - Soit d'un littéral positif et des littéraux négatifs
 - Soit d'un littéral positif
- Une clause termine toujours par un point



Syntaxe Prolog

■ Exemples de littéraux

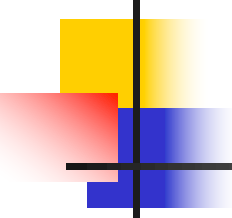
■ pere(jean, paul)

- prédicat **pere** entre les termes élémentaires **jean** et **paul** est d'arité 2
- peut être interprété par «*jean est le père de paul*»

■ habite(X, adresse(8, "rue Carnot", Dakar))

- prédicat **habite** entre la variable **X** et le terme composé **adresse(8, "rueCarnot", Dakar)** est d'arité 2
- peut être interprété par «*une personne inconnue X habite à l'adresse 8 rue Carnot, Dakar*»

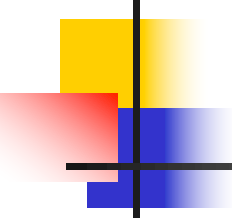
■ NB: **pas d'espace entre le prédicat et les parenthèses**



Syntaxe Prolog

- Deux types de clauses en Prolog
 - Clauses inconditionnelles : faits
 - homme(jean).
 - Clauses conditionnelles: règles
 - pere(X, Y):- parent(X, Y), homme(X).

Programmer en Prolog → construire une base de connaissance (faits et règles) pour ensuite poser des questions.



Syntaxe Prolog

■ Faits

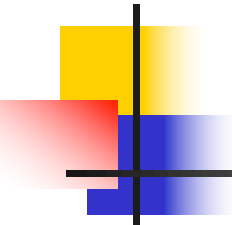
- Un fait est une règle sans prémisses
- Il est de la forme:

$F.$

où la valeur de F est vraie

■ Exemple

- $\text{pere}(\text{jean}, \text{paul}).$
- NB: une variable dans un fait est quantifiée universellement



Syntaxe Prolog

■ Règles

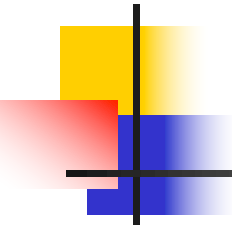
- Une règle est de la forme:

R :- **A0**, **A1**, ... , **An**.

où **A0**, **A1**, . . . , **An** sont des prédicats.

- R est vrai si les prédicats **A0**, **A1**, . . . , **An** sont vrais
- R est appelé **tête** de la clause et **A0**, **A1**, . . . , **An** **corps** de la clause
- Exemple:

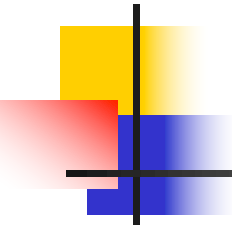
femme(X):- **personne(X)**, **sexe(X, féminin)**.



Syntaxe Prolog

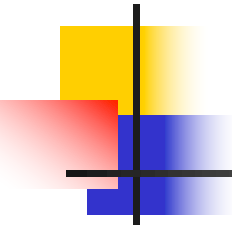
■ Règles

- Une variable apparaissant dans la **tête** d'une règle est **quantifiée universellement**
- Une variable apparaissent dans le **corps et non dans la tête** d'une règle est **quantifiée existentiellement**
- Exemple
 - `meme_pere(X, Y):- pere(P, X), pere(P, Y).`
se lit «*pour tout X et pour tout Y, X et Y sont de même père s'il existe quelqu'un qui est le père de X et le père de Y*»



Syntaxe Prolog

- Un programme Prolog est un ensemble de clauses regroupées en **paquets**
- L'**ordre** dans lequel les paquets sont définis n'est **pas important**
- Un paquet
 - Représente un prédicat
 - Ensemble de clauses dont la tête a le même symbole de prédicat et la même arité
 - L'**ordre** dans lequel les clauses sont définies est important

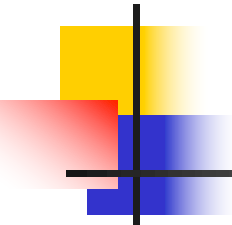


Syntaxe Prolog

- Intuitivement, deux clauses d'un même paquet sont liées par un **ou logique**
- Exemple de définition de prédicat
 - `personne(X):- homme(X).`
 - `personne(X):- femme(X).`

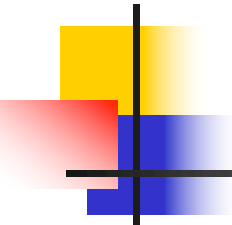
se lit X est une personne s'il est un homme ou s'il est une femme

 - Equivalent à
 - `personne (X):- homme (X) ; femme (X).`



Syntaxe Prolog - Questions

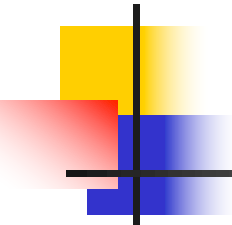
- Question en prolog
 - suite de prédicats séparés par des virgules et terminant par un point
 - La réponse prolog est **true** si la question est une conséquence et **false** sinon
 - Exemple
 - `personne(jean), homme(jean).`
`true.`
 - `personne(uasz).`
`false.`



Syntaxe Prolog - Questions

■ Question avec variable en Prolog

- Une question peut contenir des variables **quantifiées existentiellement**
- La réponse Prolog est dans ce cas **l'ensemble des valeurs des variables** pour lesquelles la question est une conséquence
- Exemple
 - `pere(jean, X), pere(X, Y).`
se lit «*existe-t-il quelqu'un dont jean est son père et qui est père d'un autre ?*»
 - La réponse de Prolog est l'ensemble des couples de valeurs de X et Y qui vérifient cette relation



Syntaxe Prolog - Questions

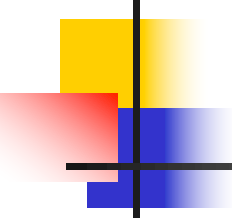
■ Exemples

■ Faits

- pere(alain, jeanne).
- pere(alain, michel).

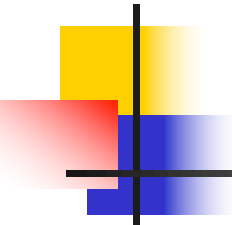
■ Questions

- pere(alain, jeanne). Réponse: true.
- pere(alain, louis). Réponse: **false**.
- pere(alain, X). Réponses: X = jeanne ;
X = michel.



Listes

- Structure de données très utilisée en Prolog
- Une liste est constituée de deux éléments:
 - Une tête
 - Une queue (liste)
- Elle est représentée par: **. (X, L)**
- La liste vide est notée []



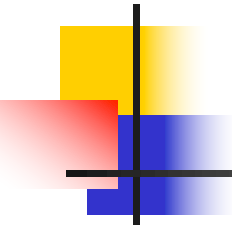
Listes

■ Notations

- $.(X, L)$ est également notée $[X|L]$
- $.(X1, .(X2, L))$ est également notée $[X1, X2|L]$
- $.(X1, .(X2, . . ., .(Xn, L). . .))$ est également notée $[X1, X2, . . ., Xn|L]$
- $[X1, X2, X3, . . ., Xn|[]]$ est également notée $[X1, X2, X3, . . ., Xn]$

■ Exemple

- $.(a, .(b, .(c, .(d, [])))) = [a, b, c, d] = [a|[b, c, d]]$



Listes

- Manipulation de listes : **appartenance**

- Définition de la fonction membre

- `membre(X, [X|_]).`
- `membre(X, [_|L]):- membre(X, L).`

- Exemple

`membre(a, [a, b, c, d]).`

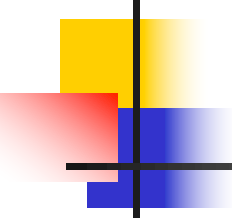
⇒ true

`membre(e, [a, b, c, d]).`

⇒ false

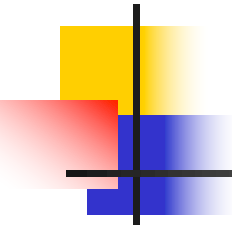
`membre(X, [a, b, c, d]).`

⇒ `X = a; X = b; X = c; X = d; false`



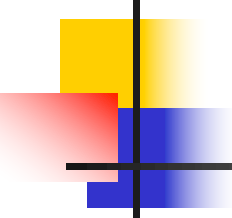
Listes

- Manipulation de listes : concaténation
 - Définition de la concaténation
 - `concat([], L, L).`
 - `concat([X|L1], L2, [X|L3]):- concat(L1, L2, L3).`
 - Exemple
 - `concat([a, b, c, d], [e, f], R).`
 - ⇒ `R = [a, b, c, d, e, f]`



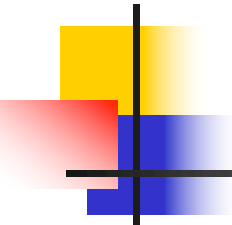
Listes

- Manipulation de listes : rang d'un élément
 - Définition de rang
 - `rang(X, 0, [X|_]).`
 - `rang(X, R, [_|L]):- rang(X, R2, L), R is R2 + 1.`
 - Exemple
 - `rang(b, R, [a, b, c, d]).`
 $\Rightarrow R = 1$



La coupure

- Elle est notée !
- C'est un prédicat sans signification logique (ni vraie ni fausse)
- Elle est utilisée pour empêcher des retour-arrière
- Elle est appliquée pour
 - La recherche de la première solution
 - Exprimer la négation
 - Exprimer la structure « si alors sinon »



La coupure

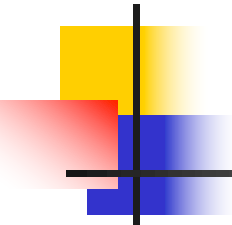
■ Exemple 1

- `personne(jean).`
 1. `homme(jean) ; succès`
 2. `femme(jean) ; echec`

■ Avec la coupure

- `personne(X):- homme(X), ! .`
- `personne(X):- femme(X), ! .`

- `personne(jean).`
 1. `homme(jean) ; succès`



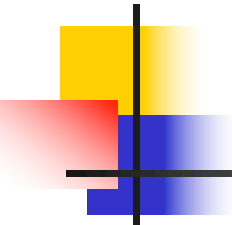
La coupure

- Exemple 2

- $\text{max}(X, Y, X) \text{:- } Y \leq X.$
- $\text{max}(X, Y, Y) \text{:- } X < Y.$

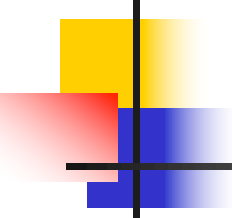
- Avec la coupure

- $\text{max}(X, Y, X) \text{:- } Y \leq X, !.$
- $\text{max}(X, Y, Y).$



Prédicats arithmétiques

- $X \text{ is } Y$: évalue Y et la valeur est unifiée à X
- $X ::= Y$: vrai si les valeurs de X et Y sont égales
- $X \neq Y$: vrai si les valeurs de X et Y sont différentes
- $X < Y$: vrai si la valeur de X est inférieure à la valeur de Y (idem pour $>$, $\geq$, $\leq$)



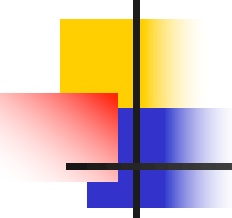
Prédicats arithmétiques

- Exemple: suite de Fibonacci

`fib(0, 0).`

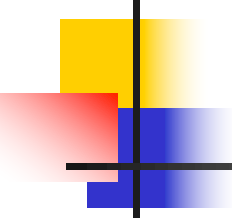
`fib(1, 1).`

`fib(N, F):- N>1, N1 is N - 1, fib(N1, F1),
 N2 is N - 2, fib(N2, F2),
 F is F1 + F2.`



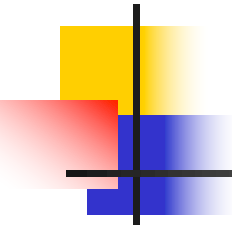
Fonctions prédéfinies

- $+$, $-$, $*$, $/$, $^$,
- `mod`, `abs`, `min`, `max`,
- `random`, `sqrt`, `exp`, `log`,
- `sin`, `cos`, `tan`, etc.



Exercice d'application

- Ecrire un prédicat qui retourne le dernier élément d'une liste.
- Ecrire un prédicat qui retourne la longueur d'une liste.
- Ecrire un prédicat qui inverse une liste.
- Définir un prédicat calculant le nième terme de la suite : $U_0 = 2$, $U_n = 2U_{n-1} + 3$



Exercice d'application (correction)

dernier(X, [X]).

dernier(X, [_|L]):- dernier(X, L).

longueur([], 0).

longueur([_|L], N):- longueur(L, N1), N is N1 + 1.

inverse([], []).

inverse([X|L], R):- inverse(L, L1), append(L1, [X], R).

suite(0, 2).

suite(N, R):- N1 is N - 1, suite(N1, R1), R is 2 * R1 + 3.